DAGSmith: Dependency-Aware Rewriting for dbt-Style SQL Pipelines

Jie Liu
jiezzliu@umich.edu
Computer Science and Engineering
University of Michigan, Ann Arbor
USA

Lin Ma
linmacse@umich.edu
Computer Science and Engineering
University of Michigan, Ann Arbor
USA

Barzan Mozafari
mozafari@umich.edu
Computer Science and Engineering
University of Michigan, Ann Arbor
USA

ABSTRACT

Modern analytics is increasingly organized as recurring SQL pipelines rather than isolated SQL statements. Tools such as dbt—which have gained extreme popularity in recent years—allow teams to write each transformation as SQL and make dependencies between transformations explicit, producing directed acyclic graphs (DAGs) with hundreds or thousands of interdependent SQL models. Existing techniques are not effective at optimizing these expensive pipelines. Traditional query optimizers and source-to-source query rewriters operate on one query at a time, while materialized-view selection and multi-query optimization address narrower forms of reuse. They do not directly exploit the pipeline-level information exposed by explicit dependencies: how intermediate results are consumed, which downstream outputs depend on each computation, where expensive work occurs relative to data reduction, which intermediate results are worth persisting, and how refresh schedules relate to the rate at which inputs change and outputs are consumed.

We introduce DAGSmith, to the best of our knowledge the first holistic dependency-aware source-to-source rewriting system for SQL pipeline DAGs. DAGSmith treats explicit dependencies as optimization signals. It analyzes each transformation together with its upstream inputs, downstream consumers, and position in the pipeline DAG, uses an LLM to propose pipeline-level refactorings, separates SQL generation and equivalence checking to reject unsafe rewrites, retunes persistence choices with a learned cost model, and selects a globally compatible set of rewrites that avoids conflicts across transformations. This enables dependency-edge simplification, non-local semantic reuse, downstream-aware pruning, pipeline-aware work placement, rewrite-materialization co-optimization, and frequency-aware optimization while keeping the pipeline executable by the same SQL engine and orchestration framework. Our extensive evaluation shows that, on the open-source Tuva dbt project, DAGSmith reduces elapsed time by 42.6% and warehouse compute cost by 67.7%; these reductions are 22.8%/83.0% larger than materialization-only tuning and 98.1%/348.3% larger than state-of-the-art single-query rewriting techniques.

1 INTRODUCTION

Rising SQL Complexity — SQL is no longer only a hand-written interface for individual analytical questions. It is increasingly the implementation language for recurring data products: governed dashboards, AI-ready feature tables, self-service BI datasets, and organization-wide reporting layers. At the same time, more SQL is produced by BI systems, code generators, and AI-assisted development tools rather than written from scratch. In fact, recent studies report that 70% of analytics professionals use AI for code development [9], and that BI-generated SQL contain extremely complex scalar expressions and relational operator trees [24]. This has created a major efficiency gap between the complexity of the SQL queries generated and what today's query optimizers are capable of effectively optimizing. Source-to-source query rewriting is the traditional way to narrow this gap: a rewriter transforms complex SQL into an equivalent form that the query optimizer is more likely to turn into an efficient plan. However, query rewriting has traditionally treated one SQL statement as the unit of optimization. Unfortunately, the effectiveness of this one-query-at-a-time approach is increasingly limited by a second trend: modern SQL is no longer produced and executed only as isolated statements, but as recurring transformation pipelines.

Rise of SQL Pipelines — Data teams are increasingly building large, recurring SQL pipelines. This growth is driven by complex business logic that must be computed repeatedly and reused consistently across governed dashboards, AI-ready feature tables, self-service BI datasets, compliance reports, and downstream applications. One of the most widely adopted tools for these workflows is *dbt* (the data build tool) [11], reported to be used by 50K+ data teams weekly across the world [8]. dbt lets teams write transformations as version-controlled SQL `SELECT` statements (called "models" in dbt, which we use interchangeably in this paper), declare dependencies among them, and compile the resulting graph into SQL jobs executed by a data warehouse. Figure 1 gives a toy example. A raw orders table feeds a cleaning transformation; the cleaned result feeds both a revenue table and a daily active-customer table. The SQL text explains each local transformation, while the edges explain how the transformations compose into a pipeline. This trend is much broader than dbt. SQLMesh [23], Dagster asset graphs [5], Airflow directed acyclic graphs [2], Prefect flows [18], Argo Workflows [3], Databricks Workflows [6], and medallion lakehouse pipelines [7] all represent recurring data transformations through explicit dependencies. In this paper, we target the SQL subset of this broader pattern, which we refer to as *SQL pipeline DAG*: a recurring directed acyclic graph whose nodes are SQL transformations or SQL-produced tables/views, and whose edges record that one node reads another node's output.

Pipeline Optimization Gap — These pipeline DAGs often include hundreds or thousands of SQL transformations that reference one another. Their scale makes them difficult to optimize with techniques designed for a single SQL statement or a flat workload: inlining every referenced transformation into each downstream query would produce deeply nested SQL with tens or hundreds of layers whose size and complexity are well beyond what today's

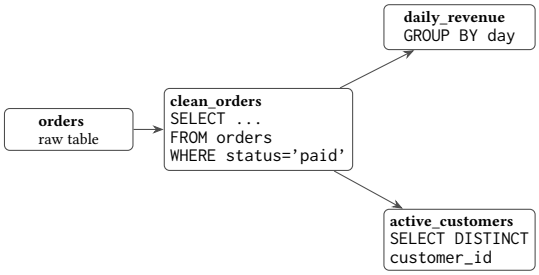

Figure 1: A toy SQL pipeline DAG. Nodes are SQL transformations or SQL-produced tables (called "models" in dbt); edges show which transformation consumes another transformation's output.

optimizers can even tackle. In particular, existing *query rewriting* techniques, whether traditional rule-based systems [26] or modern synthesis- or LLM-based techniques [10, 15, 22], rewrite one SQL statement at a time. This scope is useful for improving a standalone query, but it is too narrow for optimizations whose correctness or benefit depends on the surrounding pipeline, such as removing work no final output uses, reusing equivalent logic across separate transformations, or changing refresh frequency based on input changes and output consumption. *Multi-query optimization* techniques [19, 20] consider multiple queries together, but their classical goal is to share common scans, joins, or intermediate results across a batch of submitted queries. They do not generally rewrite a SQL pipeline by adding, removing, splitting, or merging transformations based on downstream demand and refresh behavior. *Materialized-view selection* [1, 13] chooses intermediate query results to precompute and rewrites future queries to read those precomputed results. This improves reuse when the relevant computation has already been identified, but it does not by itself determine that a pipeline should drop unused work, factor repeated logic written in different forms, move expensive operations after data reduction, or refresh different parts of the pipeline at different rates. Lastly, *Workflow schedulers* [2, 3, 5, 18] know dependencies, but usually optimize execution order, failure recovery, retries, or resource allocation rather than the SQL semantics of the transformations themselves.

Dependencies as Signals — Our key insight is that explicit dependencies are not merely execution constraints; they are signals that can be leveraged for optimization. In particular, a SQL pipeline DAG exposes six kinds of information that are hidden or much harder to recover when queries are optimized independently: **(1) Edge semantics:** producer-consumer edges reveal how one SQL transformation's output is used by later transformations; **(2) Cross-graph redundancy:** similar business logic can appear in distant parts of the graph, even when SQL text differs; **(3) Downstream demand:** downstream dependencies reveal which columns, joins, filters, and intermediate results can affect final outputs; **(4) Operator placement:** the graph reveals where expensive operations occur relative to data-reducing filters, projections, joins, and aggregations; **(5) Rewrite-persistence coupling:** unlike single-query rewriting, which is constrained to equivalent rewrites that are cheaper in isolation, pipeline rewriting can consider alternatives that produce additional reusable outputs and are therefore locally more expensive, as long as those outputs are materialized once and reused by enough downstream consumers; and **(6) Refresh patterns:** schedules and source update rates reveal how often inputs change and outputs are consumed.

Our Approach — In this paper, we introduce DAGSmith, the first (to the best of our knowledge) holistic, dependency-aware rewriting system for SQL pipeline DAGs. DAGSmith exploits the exposed dependency structure to perform six classes of optimization: **(1) dependency-edge simplification:** remove or simplify dependency edges whose purpose can be expressed more directly, while preserving final outputs; **(2) non-local semantic reuse:** factor repeated work into a shared computation across non-adjacent transformations; **(3) downstream-aware pruning:** remove work that is provably irrelevant to every final output; **(4) pipeline-aware work placement:** move expensive work after data-reducing steps as long as final outputs are not impacted; **(5) rewrite-materialization co-optimization:** decide when rewrite benefits are outweighed by materialization cost, and when materializing a rewritten result makes the rewrite worthwhile; and **(6) frequency-aware optimization:** align computation frequency with input-change rates and output-consumption rates, avoiding refreshes that do not impact consumers.

DAGSmith first analyzes compiled SQL and dependency edges to identify regions likely to contain these opportunities. It then leverages Large Language Models (LLMs) to reason over the relevant subgraph and propose concrete refactorings, but separates proposal, criticism, SQL generation, and equivalence checking so unsafe or counterproductive rewrites can be rejected before adoption. Adopted candidates are not evaluated in isolation: DAGSmith retunes persistence choices for each candidate using a learned cost model and an iterative local linearization procedure, then solves a global selection problem to choose a non-conflicting subset of rewrites. Finally, frequency information from source update rates and output demand reweights cost estimates, detects over-scheduled work, and generates splitting candidates when slow-changing logic is embedded in fast-refreshing transformations. The result is a rewritten, output-equivalent SQL pipeline that remains executable by the same SQL engine and orchestration framework, but is significantly more performant.

Contributions — This paper makes the following contributions:

(1) We formulate dependency-aware source-to-source rewriting for SQL pipeline DAGs. We present the first holistic approach to this problem (to the best of our knowledge) by identifying the pipeline-level signals exposed by explicit dependencies and the rewriting opportunities they enable: dependency-edge simplification, non-local semantic reuse, downstream-aware pruning, pipeline-aware work placement, rewrite-materialization co-optimization, and frequency-aware optimization.

(2) We present the design of DAGSmith, which combines structural graph analysis, LLM-guided refactoring, separated proposal, criticism, SQL generation, and equivalence checking, learned cost modeling, persistence tuning, frequency-aware scheduling, and global candidate selection.

(3) We conduct a comprehensive evaluation showing that, on the open-source Tuva dbt project, DAGSmith reduces elapsed time by 42.6% and warehouse compute cost by 67.7%; these reductions are 22.8%/83.0% larger than materialization-only tuning

Table 1: Optimization signals exposed by explicit SQL pipeline dependencies.

#	What explicit dependencies expose	How DAGSmith leverages them for optimization
1	Producer-consumer edges reveal how one SQL transformation's output is used by later transformations.	**Dependency-edge simplification:** remove or simplify dependency edges whose purpose can be expressed more directly, while preserving final outputs.
2	Similar business logic can appear in distant parts of the graph, even when SQL text differs.	**Non-local semantic reuse:** factor repeated work into a shared computation across non-adjacent transformations.
3	Downstream dependencies reveal which columns, joins, filters, and intermediate results can affect final outputs.	**Downstream-aware pruning:** remove work that is provably irrelevant to every final output.
4	The graph reveals where expensive operations occur relative to data-reducing filters, projections, joins, and aggregations.	**Pipeline-aware work placement:** move expensive work after data-reducing steps as long as final outputs are not impacted.
5	Dependency fanout reveals when a transformation could produce additional reusable outputs for downstream consumers.	**Rewrite-materialization co-optimization:** consider locally more expensive rewrites that pay off only when the richer result is materialized once and reused by enough downstream consumers.
6	Schedules and source update rates reveal how often inputs change and outputs are consumed.	**Frequency-aware optimization:** align computation frequency with input-change rates and output-consumption rates, avoiding refreshes that do not impact consumers.

and 98.1%/348.3% larger than state-of-the-art single-query rewriting techniques.

The rest of the paper is organized as follows. Section 2 defines the setting and illustrates the opportunity space. Section 3 presents the design of DAGSmith. Section 4 outlines the evaluation plan, Section 5 discusses prior work, and Section 6 concludes.

2 PROBLEM SETTING

Before presenting our approach, we first provide the necessary background on SQL pipeline DAGs (§2.1), then present a few motivating examples of pipeline-level optimization opportunities exposed by explicit dependencies (§2.2). Lastly, we explain why local query rewriting is insufficient for this setting (§2.3).

2.1 Background: SQL Pipeline DAGs

A *SQL pipeline DAG* is a recurring SQL program whose nodes are named transformations and whose directed edges record producer-consumer relationships among their outputs (see Figure 1 for a toy example). The DAG determines evaluation order, while node metadata can specify whether each result is materialized and how frequently it is refreshed. Because dbt calls each named SQL transformation a *model*, we use *model* for dbt-specific examples and *transformation* for the broader setting.

Target workload — Although our examples and evaluation use dbt projects, the optimization problem is more general. DAGSmith targets recurring SQL-based DAGs: pipelines in which node logic is available as SQL, edges expose producer-consumer dependencies, and system metadata records persistence and refresh behavior. dbt is a prominent instance of this setting, but similar dependency graphs arise in SQLMesh [23], Dagster [5], Airflow [2], Prefect [18], Argo [3], Databricks Workflows and Lakeflow Jobs [6], medallion lakehouse pipelines [7], and Luigi [16].

2.2 Motivating Examples

In this section, we use a small claims-processing DAG to illustrate how explicit dependencies expose optimization opportunities that are invisible when each query is rewritten in isolation. For space, the examples combine multiple opportunities in one small claims-processing DAG; in practice, each opportunity can appear independently inside much larger DAGs with hundreds or thousands of models. Figure 2(a) shows the original pipeline. Here, we use a simplified healthcare claims-processing pipeline, inspired by the open-source Tuva project [**?**]. Its main input is `claims`, a large table of claim-line records that is refreshed frequently. It feeds into service-category models such as `dialysis`, `psychiatric`, `equipment`, and `nursing`. These models classify claim lines using SQL predicates over claim attributes; Figure 3 gives simplified examples. The `nursing` model excludes claim lines already categorized as equipment, so it reads `equipment` as an exclusion list. The `claims_summary` model joins claim lines with a slow-changing lookup table, `code_catalog`, that maps raw claim codes to descriptions, and then applies code-grouping logic over those descriptions. We next describe three examples that exploit these explicit dependencies; when all applied together, they produce the more efficient DAG in Figure 2(b).

Example 1: Cross-graph redundancy + rewrite-persistence coupling. In Figure 2(a), the fanout from `claims` to `dialysis`, `psychiatric`, and related service-category models exposes cross-graph redundancy: several models scan the same large source, namely `claims`, and evaluate related billing-code, taxonomy-code, or diagnosis-code predicates in separate SQL files. By exploiting this redundancy, one can create the shared `service_flags` model in Figure 2(b) and rewrite downstream models such as `dialysis` to read the corresponding flag, as shown in Example 1 of Figure 3 (*non-local semantic reuse*). This scenario also allows for another optimization: the shared model can compute additional reusable flags, which is worthwhile only if the richer result is materialized once and reused by enough downstream models (*rewrite-materialization co-optimization*).

Example 2: Edge semantics + downstream demand. We can also exploit the edge semantics in Figure 2(a): the edge from `equipment` to `nursing` says that `nursing` consumes `equipment`, but the SQL definitions show that `nursing` uses `equipment` only to exclude claim lines whose category is equipment. This exclusion can be expressed directly as a predicate over `claims`. By exploiting this edge semantics, one can rewrite `nursing` as shown in Example 2 of Figure 3, removing

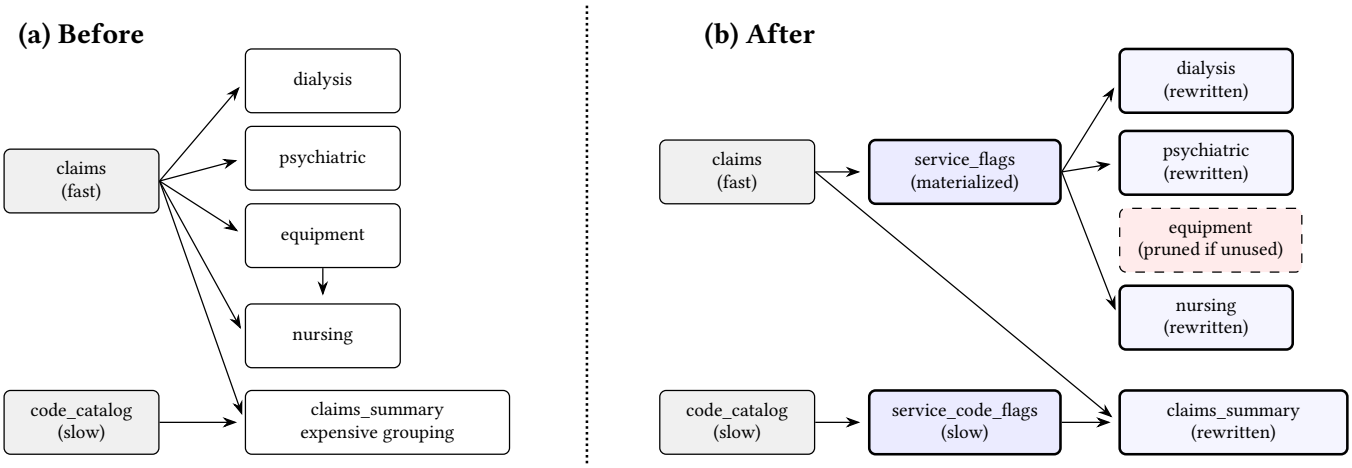

Figure 2: Healthcare SQL pipeline DAG used for three motivating examples. The dotted line separates the original graph (a) from the cumulative rewritten graph (b). The rewritten graph introduces reusable service flags, rewrites downstream service-category models to consume those flags, removes the `equipment` dependency from `nursing`, and moves slow-changing reference-code grouping out of the fast claims path.

Ex. 1: service flags

```sql
-- dialysis model before
SELECT DISTINCT claim_id
FROM claims
WHERE bill_code LIKE '72%'
   OR taxonomy IN (...);

-- new service_flags model
SELECT claim_id,
  MAX(bill_code LIKE '72%'
      OR taxonomy IN (...)) AS f_dia,
  MAX(diagnosis BETWEEN '290' AND '319') AS f_psy
FROM claims;

-- rewritten dialysis model
SELECT claim_id FROM service_flags
WHERE f_dia;
```

Ex. 2: nursing/equipment

```sql
-- nursing model before
SELECT * FROM claims c
WHERE c.facility_code IN ('31','32')
AND NOT EXISTS (
  SELECT 1 FROM equipment e
  WHERE e.claim_id = c.claim_id);

-- rewritten nursing model
SELECT * FROM claims c
WHERE c.facility_code IN ('31','32')
  AND c.category <> 'equipment';

-- equipment model has no remaining consumers
```

Ex. 3: slow references

```sql
-- claims_summary before
SELECT c.id, group_code(r.description) AS grp
FROM claims c JOIN code_catalog r USING(code);

-- new service_code_flags model
SELECT code, group_code(description) AS grp
FROM code_catalog;

-- rewritten claims_summary model
SELECT c.id, f.grp
FROM claims c JOIN service_code_flags f USING(code);
```

Figure 3: SQL fragments for the healthcare examples in Figure 2. Example 1 creates a new `service_flags` model and rewrites concrete downstream models such as `dialysis`. Example 2 rewrites the `nursing` model and may leave `equipment` with no consumers. Example 3 creates a slow `service_code_flags` model and rewrites `claims_summary` to join it.

the edge from `equipment` to `nursing` in Figure 2(b) (*dependency-edge simplification*). The graph also exposes downstream demand: if no remaining output consumes `equipment`, the model can be removed (*downstream-aware pruning*).

Example 3: Refresh + placement. The `claims_summary` model in Figure 2(a) combines fast-changing claims with slow-changing reference codes and then applies code-grouping logic after the join. The graph exposes refresh patterns, because `claims` and `code_catalog` change at different rates, and operator placement, because the grouping occurs after reference data has been joined into the high-volume claims stream. By exploiting these signals, one can create `service_code_flags` on the slow reference side and rewrite `claims_summary` to join that precomputed result, as shown in Figure 2(b) and Example 3 of Figure 3 (*frequency-aware optimization* and *pipeline-aware work placement*).

2.3 Why Local Rewriting Falls Short

The three examples above are not isolated curiosities; they illustrate a general capability gap. Single-query rewrite systems [22, 26] operate within the boundary of one query and cannot restructure across transformation files. Multi-query optimization and common subexpression elimination [20] share intermediate computations across queries but rely on syntactic matching, so they miss the cross-transformation semantic equivalence in Example 1. Materialized view selection [1] chooses among existing view candidates and treats update cost as a static parameter rather than as a structural driver of DAG design (Example 3). Code clone detection and lineage tools can identify structural similarity or trace data provenance, but they do not perform the optimization itself.

The gap is not that any one of these techniques is poorly designed; it is that none of them targets the setting we are concerned with. Optimizing a SQL pipeline DAG requires an optimizer that can combine the six capabilities above: use producer and consumer

context, find semantic reuse beyond syntactic overlap, prune work that no downstream output consumes, move expensive work after data-reducing steps, evaluate rewrites together with persistence choices, and account for recurring refresh behavior. The remainder of this paper presents an approach that integrates these capabilities into a coherent optimization framework.

3 APPROACH

lin: IMPORTANT: We need to double check all the references to the "opportunites" in this section. It seems that Barzan has removed the numbered opportunites in Section 2.2, but we still have the six "optimizations"in Section 1. So we can probably just reference these **optimizations** by number from Section 1. But double check everything to make sure that the numbers are consistent.

DAGSmith optimizes a SQL pipeline DAG along three interacting dimensions: graph refactoring, materialization, and refresh frequency. Each dimension leverages a group of the opportunities from Section 2.2: refactoring restructures the DAG to share or remove work (opportunities (1)–(4)), materialization decides which results are persisted and composes overlapping rewrites (opportunity (5) and global rewrite composition), and frequency aligns each node's execution rate with how often its inputs change (opportunity (6)). These dimensions are coupled: the best choice along one depends on the others, so they should not be optimized independently. We quantify these interactions with an ablation study in Section 4.3.

DAGSmith realizes these dimensions in the pipeline of Figure 4. Cheap structural analyses first scan the whole DAG and rank model groups by a fingerprint-based score that estimates the benefit of refactoring them jointly (Section 3.1). The top-ranked groups are passed to an LLM, which reasons over each region and proposes one or more concrete refactorings (Section 3.2). The structural analyses decide where in the DAG to refactor; the LLM decides what refactoring to perform. Every refactoring is adopted only after it passes equivalence checks that the LLM writes for itself and that are tested against the project's real data. A refactoring's benefit depends on how its nodes are materialized, so DAGSmith tunes each candidate's materialization with a learned cost model and selects a compatible subset to adopt (Section 3.3). The frequency dimension cuts across these stages (Section 3.4): it reweights the cost-driven decisions, removes refreshes that recompute unchanged data, and contributes structural splitting candidates to the first stage.

3.1 Candidate Identification

Candidate identification finds where in the DAG the refactoring opportunities (1)–(4) of Section 2.2 are worth pursuing. Refactoring reasons jointly about a region of interdependent models, so DAGSmith confines it to small, bounded regions rather than the whole project. Since most regions are not worth refactoring, DAGSmith first scans the whole DAG with cheap structural analyses that rank regions by likely benefit, reserving the expensive refactoring step (Section 3.2) for the few that pay off.

The analysis must decide when two models perform the same expensive work, and the natural ways of testing for this fall short. For example, matching on SQL text fails as soon as the same logic is written differently. Matching on plan structure goes further—multi-query optimization and view-matching already see through join order, predicate order, and aliasing—but it reasons about one compiled query at a time and treats a referenced model as an opaque input. Therefore, it cannot recognize work shared across model boundaries. Figure 7 shows the problem: Models A and B compute the same join, but Model A filters its input inline while Model B reads the already-filtered input from a sibling model it references. On the compiled SQL, a plan-subtree matcher therefore sees two unrelated trees over different tables. DAGSmith instead matches on the operation each model computes through dataflow analysis, resolving references so that a model boundary no longer hides shared work, and reduces the two to a single reuse candidate—which the LLM and a later equivalence check confirm is safe to share. Two analyses build on this: reuse finds repeated expensive work to share (opportunity (2)), and prune finds work that no downstream model consumes or that runs too early (opportunities (1), (3), and (4)). Only the highest-scoring regions are passed to the LLM.

Dataflow representation. Both analyses run on a *dataflow representation* rather than on SQL text: DAGSmith compiles each model into a tree of typed operators and resolves every column and `ref()` back through the models that produced it. Figure 5 shows why this is needed. Model B reads the sibling R_p, so at the SQL level its input is an opaque table name; resolving the reference substitutes R_p's definition, the filtered R, and reconstructs the join B actually computes over R and S. Model A reaches the same operation by filtering R inline. The two resolved trees coincide even though the `ref()` boundary hides the correspondence in the source, so the analyses can compare models by what they compute rather than by how the SQL is split across the DAG.

Reuse analysis. Reuse analysis summarizes each model's dataflow representation by a *signature*: its expensive operations (joins and group-by aggregations), the keys they operate on, and their resolved inputs. Models whose signatures match are grouped into one candidate and scored by the cost of the shared operation and the number of models that share it. A high score is a *targeting signal*, not an instruction to materialize a shared subexpression; it marks where the LLM should look.

Prune analysis. Prune analysis targets joins in two forms. A *redundant* join produces a result no downstream model consumes, confirmed by tracing column lineage from the leaf models, and can be removed outright or, when it is a semi-join whose filter is already derivable from the probed relation, collapsed to a predicate. A *mistimed* join is correct but computed too early, inflating an expensive downstream operation whose result does not depend on it; pushing it past that operation avoids carrying its product through the work. Both targets are different from local dead-code elimination in scope: whether a join's result is used, or used too soon, is a fact about the rest of the pipeline that a single-query rewriter cannot see.

From candidates to subgraphs. lin: How the groups are ranked? I.e., how do you decide the score? I think we need one sentence about it (doesn't need to be long). The two analyses produce a ranked stream of candidate model groups. For each, DAGSmith extracts a subgraph of the candidate models plus a bounded neighborhood of upstream and downstream context, enough for the next stage to reason about correctness; groups whose neighborhoods exceed the bound are skipped. lin: Just need a few words about how

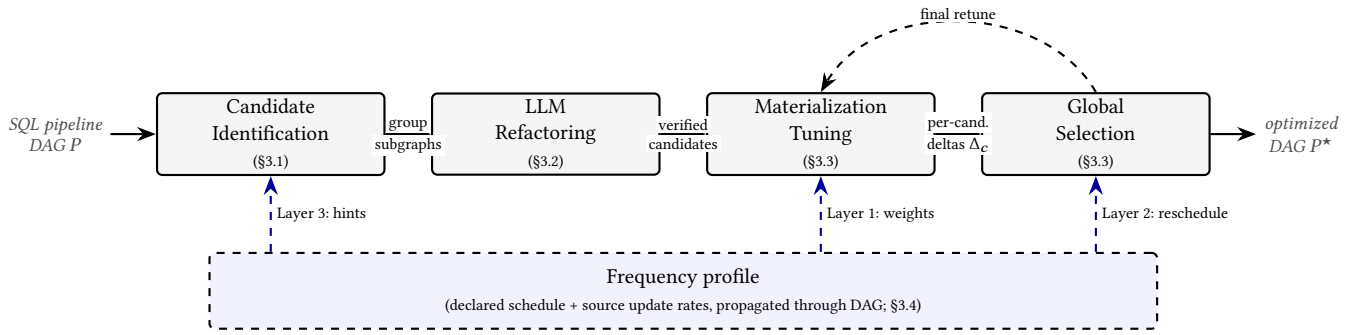

Figure 4: End-to-end pipeline of DAGSmith. The top row shows the four main stages and the artifacts that flow between them; after global selection, a final materialization pass retunes the combined DAG. The dashed band underneath is the frequency dimension (Section 3.4): a single frequency profile feeds three of the four stages, contributing structural splitting hints, cost weights for materialization tuning, and schedule corrections at selection. lin: refrence the sections explicitly (e.g., Section 3.1) in the figure instead of using §, since we use the full reference throughout the paper.

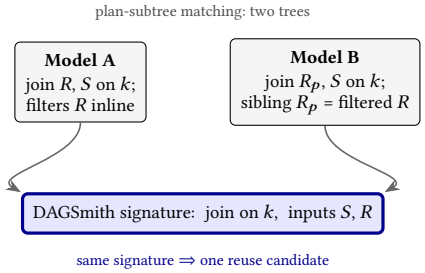

Figure 5: Candidate identification discovers shared computation across models based on dataflow, not plan shape. Models A and B compute the same join on k, but Model A filters R inline while Model B reads the already-filtered R from a sibling model R_p; on the compiled SQL the two are different trees over different tables, while DAGSmith resolves the reference and reduces both to one signature and a single reuse candidate.

the contenxt is extracted. 1-hop models? 2-hop? Etc. The next subsection describes how the LLM turns these subgraphs into concrete refactorings.

3.2 LLM Refactoring

This stage produces the refactorings that realize opportunities (1)–(4) of Section 2.2. As motivated in the overview, producing these cross-boundary, semantic rewrites is a reasoning task rather than a pattern-matching one, which is why DAGSmith uses an LLM rather than a fixed rule set. But asking an LLM to rewrite SQL that will run against a production warehouse raises three problems: the rewrite may be *slower* than the original, may be *incorrect*, or may be *too expensive to generate* when a region contains many models.

DAGSmith addresses all three by splitting rewriting into four roles, each a separate call with one job: a *proposer* reads the region and lists candidate optimizations, an adversarial *critic* rejects proposals that would slow the pipeline or silently change its results, a *rewriter* turns the survivors into concrete SQL, and a *verifier* tests each rewrite against the warehouse's real data before it is trusted. Separating proposal from criticism keeps a single prompt from having to be both creative and skeptical, and grounding acceptance in real data rather than in the model's own assurance is what makes adopting LLM-written SQL safe.

Proposal. The first call ingests the subgraph's SQL and dependency edges and produces a domain-level reading of the region: a short semantic summary of each model and the end-to-end business question the subgraph answers. This reading is what lets later stages recognize equivalent computations written in dissimilar SQL. The call then outputs a list of *opportunities*, each describing the work it would remove or shrink and how—proposals to be rewritten and checked later, not finished rewrites.

Filtering. A second call acts as an adversarial critic, going through the proposals and trying to break them. We keep this separate from proposal on purpose: a single prompt asked to both propose and doubt does neither well, and in practice the proposing voice often wins out. The critic looks for two failure modes. The first is a slowdown—a proposal that looks like reuse-driven savings but adds a write and a scan without cutting downstream work. The second is a correctness trap, shown in Figure 6: the proposal drops a join by reading a tag from a sibling deduplicated to one row per item, silently discarding the secondary tags the original join admitted. The critic returns one of three verdicts—*approve*, *reject*, or *conditional*—where a conditional verdict records a data property the rewrite must satisfy, carried into the next stage.

Original: join admits all tags

```sql
SELECT i.item_id,
  MAX(CASE WHEN t.tag='A'
    THEN 1 ELSE 0 END) AS flag_a
FROM {{ ref('items') }} i
JOIN {{ ref('item_tags') }} t
  USING (item_id)
GROUP BY i.item_id
```

Proposal: drop join, read tag

```sql
SELECT s.item_id,
  CASE WHEN s.tag='A'
    THEN 1 ELSE 0 END AS flag_a
FROM {{ ref('items_staged') }} s
-- items_staged keeps one tag/item;
-- secondary tags are silently lost
```

Figure 6: Correctness trap rejected by the proposal critic. The original aggregates over all tags per item; the proposal reads from a sibling that has been deduplicated to one tag per item, and silently drops the rest.

Rewriting. The opportunities that survive filtering, together with the analysis and the SQL of the affected nodes, go to a third call that produces concrete refactored SQL: it may rewrite nodes, mark nodes as removed, or introduce new shared nodes. Conditional opportunities carry their recorded constraints into this prompt. A node with consumers outside the subgraph may be rewritten internally but must produce identical output rows, since its contract with those consumers is fixed.

Equivalence verification. Filtering judges proposals one at a time, but the rewriter often bundles several into a single change—and a trap that the critic would have rejected on its own can slip back in once it is buried inside a larger rewrite. The grain-collapse shortcut of Figure 6, for instance, can resurface inside a consolidated aggregation the critic approved on other grounds. Because a bundled rewrite can be wrong even after filtering, DAGSmith checks each one against real data. A fourth call, the *correctness critic*, writes a check for every adopted opportunity: a SQL query that returns rows only when the rewrite has changed the data. These run against the target warehouse under a per-query byte cap, so any row that comes back is a real counterexample. When a check fails, the violated invariant goes back to the rewriter, which must either find a different strategy that respects it or revert to the original. The loop stops once every check passes, the LLM has no new strategy to try, or the per-group attempt budget runs out—in which case the group keeps its original SQL and contributes nothing.

Model families and propagation. A subgraph often contains a *family* of near-duplicate models that apply the same operation to different data subsets, and running the full four-call sequence on each would be wasteful. DAGSmith instead runs it once on a representative and propagates the accepted rewrite to the rest with a lighter per-member call. Where a project has no family structure, the sequence runs over every model unchanged.

3.3 Materialization Tuning

This stage takes up two opportunities from Section 2.2: rewrite-materialization coupling (5) and global rewrite composition. lin: what is the second opportunity? Does seen to have this term in Section 2.2. A refactoring's benefit cannot be determined in isolation: whether it helps depends on which of its nodes are stored as tables and which are inlined as views, and the best choice differs from one candidate to the next, so a candidate must be scored together with its own best materialization. Even then, a second challenge remains: candidates overlap and cannot all be adopted at once, so they must be composed into a compatible set. This subsection addresses both—tuning one candidate's materialization, then selecting a compatible set.

Tuning one candidate. Finding a candidate's best materialization is hard because a node's materialization is not a local choice. A table acts as a barrier: it is built once, and its consumers read its stored rows. A view does not; a query that reads a view re-executes the view's SQL, expanding recursively through any parent that is itself a view until it reaches a table or source. Turning a node from table to view therefore removes its build cost but pushes its computation, along with the chain of views above it, into every query that reads it, once per consumer execution. A node's cost is set by a window of the DAG around it, and because those windows overlap, the choices are coupled: the marginal value of one decision depends on the rest of the assignment. Total cost is not a sum of per-node costs, which rules out tuning node by node, and the joint space is far too large to enumerate. lin: What does this "cost" refer to? Where is it used? In general, I couldn't really follow the discussion here... I think we're missing the intuitive discussion (2-3 sentences) on at a high-level how our method works and why it's better. We use this learned cost model, but why? What does it do? Why it's better? We use this linear programming for each, but why? What does it do? Why it's better? Overall, what's our key methodology? You just need a few sentences to establish these, but without them, it's very difficult for me to follow the text below and understand why I'm reading it. You added this for Section 3.1 and 3.2, which helped a lot.

Two different costs stand in the way, and DAGSmith addresses them separately. lin: Are these two "costs" the same as the "cost" above? I'm confused... The first is evaluation: scoring a configuration would normally mean executing it. A learned cost model, trained on instrumented runs of the original pipeline, predicts the cardinality and slot time of every node under a given assignment, which prices a whole configuration without running it; the new nodes a refactoring introduces, having no measurements, are predicted the same way from their SQL and their parents' cardinality. lin: This "cost model" part is also a bit vague. What's the input of this model and how do we featurize it? What's the model algorithm (doesn't need the detailed equation, but we don't even now which ML model do we use... It's not going to be super accurate either, right? So what do we do if it's not accurate? We don't need all the details but we need some basic understanding of it.)

The second is search. lin: What is the goal of the search? Why do we need it? I don't think we have established it... DAGSmith navigates the space with **iterative local linearization**, a successive-approximation scheme that replaces the true nonlinear cost with a local linear surrogate, solves it, and re-linearizes at the result. Concretely, at iteration k, holding the current assignment $t^{(k)}$ fixed, DAGSmith uses the cost model to measure two local quantities: a baseline build cost $B_j^{(k)}$ for each node j (its direct parents forced to tables) and an edge-local delta $U_{ij}^{(k)}$, the extra cost at j when only parent i is inlined as a view. Treating these as constants for the iteration, it solves a small ILP for $t^{(k+1)}$:

$$\min_{t,\,y} \sum_{j \in V} t_j \, B_j^{(k)} \;+\; \sum_{i \to j \in E} y_{ij} \, U_{ij}^{(k)}, \qquad \text{s.t.} \quad y_{ij} = (1 - t_i) \wedge t_j,$$

where y_{ij} is 1 exactly when node j is built and reads a view-parent i, and a trust-region cap on flips keeps each step where the coefficients hold. Because flipping a node changes the cardinalities its descendants see, B and U are recomputed after each move and the step repeated until the assignment stops changing; an effect the current coefficients miss, such as the cost of also flipping a sibling, then surfaces in the next round. The loop converges to a local optimum rather than a global one. This is safe because the unrefactored DAG is itself scored as a baseline candidate: a candidate that lands on a poor assignment loses to it and is not adopted; imperfect search costs a missed opportunity, never a regression below the original.

A candidate's *delta* is the cost of the original under its best materialization minus the candidate's under its own: the reduction

the refactoring delivers when adopted alone. lin: I'm also losing a bit of context here. Why we need this delta? What's its purpose? It's a bit abrupt here.

Selecting a compatible set. Each delta is measured as if its candidate were adopted alone, but the candidates cannot simply be unioned. Two candidates conflict when they rewrite, create, or remove any of the same models, and within one region the LLM may offer several variants of which at most one is adoptable. DAGSmith selects the subset that maximizes total delta subject to these constraints using a small integer program with an off-the-shelf solver. Because each delta was measured under the candidate's own materialization, DAGSmith then retunes the combined project once more, since the best assignment for the chosen combination need not match any single candidate's.

3.4 Frequency-Aware Optimization

This stage takes up opportunity (6), frequency-aware optimization. The stages so far treat each node's slot time as a fixed per-execution cost. In a recurring pipeline that is suboptimal: a node's real overhead is its per-run cost multiplied by how often it must actually run. Treating cost as per-execution therefore overvalues savings on rarely-run models and undervalues them on frequently-run ones. DAGSmith adds a frequency dimension to correct this. Effective frequencies are derived from the declared schedule at sink nodes and the observed update rates at source tables, propagated through the DAG, and applied at three layers; the first two leave the DAG structure intact, while the third feeds a new class of candidates back to candidate identification (Section 3.1).

Layer 1: Frequency as cost weight. Materialization tuning and selection (Section 3.3) minimize cost as if every model ran at the same rate. DAGSmith reweights their cost-model inputs by each node's effective frequency before the decisions are made; nothing else about them changes.

Layer 2: Reconciling scheduled and effective frequency. A model's effective frequency is the rate at which it produces *new* output, the slower of how often its consumers need fresh output (f_j^{demand}) and how often its inputs actually change (f_j^{data}):

$$f_j = \min\left(f_j^{\text{demand}},\ f_j^{\text{data}}\right).$$

Where the schedule fires faster than f_j, the model re-executes and reproduces the output it produced last time. DAGSmith flags such over-scheduled models and reduces their schedule to f_j. This changes neither SQL nor materialization, only the schedule, yet every redundant execution it removes is eliminated outright.

Layer 3: Frequency-driven splitting hints. The first two layers leave the DAG structure intact; the third changes it. When a model has parents that change at different rates, it inherits the fastest, so any work inside it that depends only on slow-changing parents is recomputed at the fast rate even though its inputs have not changed. DAGSmith identifies such models and emits the location to candidate identification as a hint, alongside the reuse and prune signals. The LLM then proposes a frequency split of the kind in Example 3 from Section 2.2: the slow-side work is extracted into its own slow-cadence model and the original becomes a lightweight join, after which the candidate flows through the same filtering, rewriting, verification, and selection as any other.

4 EVALUATION

We evaluate DAGSmith on real dbt projects, organized around three questions:

(1) How does DAGSmith compare to existing baselines for SQL pipeline optimization, and what classes of optimization account for the gap?
(2) How much does each component of DAGSmith's architecture contribute to the overall gain? We ablate the three optimization dimensions (refactoring, materialization, and frequency) and the dataflow-driven targeting that feeds the LLM stage.
(3) Is the underlying machinery sound and affordable? We measure the effectiveness of the equivalence-verification loop at catching unsafe rewrites, the LLM cost of running the system end-to-end, and the accuracy of the cost model that drives materialization decisions.

Summary of results. On the open-source Tuva and Stripe dbt projects, DAGSmith reduces BigQuery slot-time, the unit billed under capacity pricing, by 51.1% and 47.4% to build the pipeline once, and by 67.7% and 54.8% in recurring cost averaged across three representative refresh schedules per project. Both figures beat the strongest baseline by a wide margin (on Tuva, 67.7% against its 37.9% recurring-cost reduction), because DAGSmith reaches cross-model refactoring, materialization revision, and frequency-aware rescheduling that single-query and syntactic cross-model rewriters cannot. An ablation confirms the three dimensions are complementary, and that dataflow-guided targeting spends a fixed LLM budget more effectively than random subgraph selection (51.1% vs. 43.1% single-run). The supporting machinery is sound and affordable (Section 4.4): the verification loop reliably rejects unsafe rewrites, the LLM cost is paid once and amortized over many runs, and the learned cost model is accurate enough for the decisions it drives.

4.1 Experimental Setup

Workloads. We evaluate on two real dbt projects. *Tuva* is an open-source dbt package for healthcare data transformation. The full project comprises 842 SQL models organized across 17 modules. We use the claims preprocessing module, which contains 255 models (about 30% of the project) and is the subset whose models exhibit substantial computational complexity. We run it on a synthetic patient population of 10K, with the largest source table at 505 MB. *Stripe* is a payments-analytics project of 65 models, already factored by hand into intermediate stages so that the easy syntactic wins have been taken. We run it with the largest source table at 199 MB. The two projects together cover the range we want to evaluate: a large project that has accumulated cross-model patterns from multiple analysts, and a smaller, already-factored project where remaining gains must come from materialization, frequency, or semantic restructuring beyond syntactic deduplication.

Testbed. Experiments run on BigQuery with a 100-slot reservation on the Standard edition under capacity-based pricing. DAGSmith is not specific to BigQuery: it optimizes SQL pipelines in general,

Table 2: Frequency profile averages per project. Each row is the fraction of root and leaf models at each cadence, averaged across three profiles.

	Roots			Leaves		
Project	wk	day	hr	wk	day	hr
Tuva	0.18	0.57	0.25	0.07	0.20	0.73
Stripe	0.13	0.60	0.27	0.05	0.52	0.43

and we use BigQuery because our open-source workloads are built on them. We report slot-time as the primary cost metric, since it is the billing unit under capacity pricing, and elapsed time as a secondary latency metric. Each reported number is the median of three runs. To simulate realistic production schedules, we use three frequency profiles per project that assign root and leaf models to hourly, daily, or weekly cadences, with interior models derived by propagation. lin: How these frequencies are assigned within roots or leaves? Like which model is weekly and which model is dayly? just random? Table 2 summarizes the average cadence mix per project; all frequency-weighted results in this section are averaged across the three profiles.

Cost metrics. We report two views of cost. *Single-run cost* is the cost of executing the full DAG once. *Frequency-weighted cost* weights each model's per-execution cost by the rate at which it actually needs to run, summed over the DAG; under capacity pricing this corresponds to the project's billed cost over a period. Refactoring and materialization affect both metrics; frequency optimization affects only the frequency-weighted metric. Improvements are reported as percent reduction relative to the original project.

Baselines. We compare against three baselines spanning the design space of SQL pipeline optimization.

LearnedRewrite [28] is a rule-based per-query rewriter. It applies a portfolio of equivalence-preserving transformations to each model's compiled SQL independently, guided by a learned policy over which rules to apply and in what order. It is competitive on standalone analytical queries but has no view of the surrounding DAG.

GenRewrite [15] is an LLM-based per-query rewriter. It introduces natural-language rewrite rules that transfer knowledge across queries and a counterexample-guided loop that iteratively corrects syntactic and semantic errors in the rewrites. Like LearnedRewrite, it operates one model at a time.

CSE is a cross-model common subexpression baseline [**?**], lin: broken sitation. lin: citation still broken when I compile it... which detects subexpressions shared across a SCOPE script's stages and materializes each once, re-optimized at the query optimizer's memo. The memo step is not portable, since managed warehouses do not expose their optimizer's internal state to external tools. We adapt the approach by detecting shared subexpressions at the AST level and selecting which to materialize heuristically. CSE is the strongest available position for a syntactic, cross-model approach on a managed warehouse. lin: is "strongest available" the correct way to phrase this? "strongest" according to what standard?

Table 3: Improvement over the original project (% reduction; higher is better). Medians of three runs on a 100-slot BigQuery reservation.

	Single-run		Freq-weighted	
	Elapsed	Slot	Elapsed	Slot
Tuva (255 models)				
LearnedRewrite	−19.6	−2.5	−20.2	−3.0
GenRewrite	9.7	17.7	21.5	15.1
CSE	12.2	34.6	27.7	37.9
DAGSmith	**30.9**	**51.1**	**42.6**	**67.7**
Stripe (65 models)				
LearnedRewrite	2.6	2.7	−0.5	4.5
GenRewrite	7.6	7.1	1.1	9.3
CSE	−3.2	−5.6	−3.1	−0.6
DAGSmith	**40.0**	**47.4**	**36.1**	**54.8**

4.2 Comparison with Baselines

Table 3 reports each method's improvement over the original project on Tuva and Stripe; DAGSmith leads on every metric on both. The gap between the single-run and frequency-weighted columns is the contribution of frequency optimization, which affects only the frequency-weighted view.

Each baseline falls short for a different structural reason. We name the limit in each case, then summarize the classes of optimization that account for the gap above the strongest baseline.

LearnedRewrite: rules tuned for standalone queries do not generalize to dbt models. LearnedRewrite applies equivalence-preserving rules independently to each model. lin: Need a bridge to say this will miss many optimizations, such as XYZ discussed in Section XXX (reference some optimizations or examples here. Then, say for example, on Tuva XYZ. Then on Stripe XYZ.) On Tuva, the one rule that fires at scale rewrites `SELECT DISTINCT ... JOIN` into a join of pre-deduplicated inputs; it wins on near-unique keys and loses on coarse ones, where the forced grouping pass yields a worse plan than the warehouse planner would, and the two effects cancel. On Stripe the triggering shape is largely absent. lin: ← I don't understand this sentence... Both headlines are essentially flat lin: what does headline means here?: rule-based per-query rewriting has little left to do once a planner has handled the easy cases and cross-model structure is out of reach.

GenRewrite: per-model LLM reach helps where there is slack, hurts where there is not. GenRewrite finds per-model algorithmic rewrites a rule-based system cannot, such as replacing $O(n^2)$ self-joins with $O(n)$ window-function plans; on Tuva these account for most of its 17.7% slot reduction. lin: That's still not very good, right? So could you just add one sentence to summarize why? On Stripe it gains far less (+7.1% slot) and a single-run reading turns net negative: with no signal that nothing needs changing, it rewrites the largest models with projections the planner already applied for free and disrupts plans the warehouse had optimized. lin: ← I don't understand this sentence... An LLM rewriter operating one model at a time helps where there is algorithmic slack a planner cannot resolve and hurts where there is none. lin: Why it has to hurt? It may not help, but it's not very clear it necessarily hurts

the performance. You could say it "may hurt", but you also need an intuitive explanation.

CSE: syntactic matching sees duplication, not equivalence. CSE detects subexpressions shared across models at the AST level and is the only baseline with cross-model scope. It is strong where the same SQL recurs, collapsing Tuva's largest classifier family from 6,135 s to 613 s on syntactic match alone, and weak elsewhere: models that compute the same operation over different inputs share the work in substance but not in SQL syntax, which CSE cannot match. On Stripe, where a human engineer has already taken the easy syntactic wins, it has almost nothing to do. lin: This seems very strong... You want to phrase it objectively. The tone sounds a bit negative... Also, it actually hurts a little bit on Stripe, right? So it's interesting to have 1 sentence to comment on why.

DAGSmith: four patterns that the baselines structurally cannot reach. DAGSmith's gap above the strongest baseline comes from four recurring patterns: the first two are cross-model refactorings that go beyond syntactic matching, the next two exercise dimensions outside SQL refactoring alone. We illustrate each with one concrete instance from the workloads.

(1) Semantic-equivalence factorization across dissimilar siblings. Models that compute the same logical operation over different inputs are folded into one shared intermediate, even with no syntactic overlap. On Tuva, six inpatient encounter-rollup models compute the same per-encounter aggregations over different encounter types; DAGSmith synthesizes one rollup that carries the encounter type as a column and replaces all six bodies with lookups. The family drops from 914 s to 211 s, where CSE recovers only 127 s. lin: what does recovers 127s mean? Also, isn't 127 smaller than 211, so faster? AST matching cannot reach this, but the equivalence is clear once the LLM stage reads each model.

(2) Cross-model narrowing. When many consumers each scan a wide table to read a small subset of its columns, the projection is extracted into a narrow intermediate they all read. On Tuva, a 14-branch union consumer reads one column from a wide claim-line table per branch; once DAGSmith extracts the narrow projection, the consumer drops from 1,420 s to 374 s. The shared work is column-level, which per-query rewriters do not see and AST matching misses because each branch references a different upstream model. lin: why the AST match misses the column-level shared work? It's not very intuitive here.

(3) Materialization revision unlocks planner reach. A model materialized as a table is an opaque scan to the planner. In contrast, as a view, the planner sees the source SQL and can push predicates, prune columns, and reorder joins across the staging boundary. DAGSmith treats materialization as an optimization variable and revises it across the project. On Stripe, flipping a wide reporting model's staging layer from tables to views cuts its bytes scanned by a third and its slot by 2.5×, on byte-identical SQL. lin: what does byte-identical SQL mean? Also, why changing from table to view reduces scan? If we materialize as table, wouldn't subsequent models scan less? The savings come entirely from the planner but exist only because DAGSmith changed the materialization choice, which no approach operating on compiled SQL alone can reach.

(4) Materialization revision tied to refactored structure. The right materialization for the original project is not the right one for the refactored project: once a refactoring changes which models read from which, an upstream table's build cost can stop being worth paying. On Tuva, six voting and normalize stages, each feeding a single consumer, were tables in the original project; once DAGSmith refactors the consumers, the tuner flips them to views and the family's slot drops by 24%. lin: Again, why flipping from table to view drops slot? CSE, which reuses the original project's materialization choices, regresses on the same family. lin: then what? Did DAGSmith fix this? Or we just regress? Neither pass alone finds the configuration the joint pass does, direct evidence that refactoring and materialization must be tuned jointly.

Frequency optimization accounts for the gap between the single-run and frequency-weighted columns of Table 3: DAGSmith reduces the firing rate of models whose inputs change less often than they execute, eliminating redundant runs outright. lin: there are other layers in frequency-aware optimization, right? Did they help? I think we want to say a bit more here, since we have a full subsection on this optimization Like materialization, it is invisible to any approach that operates on compiled SQL alone.

The two projects exercise these patterns in different proportions: Tuva's gap above the strongest baseline is dominated by patterns 1 and 2 (cross-model refactoring), Stripe's by pattern 3 (materialization revision). The same pipeline produces both because the optimizer applies what fits the project in front of it rather than committing to one mechanism upfront. The next subsection quantifies the contributions component by component.

4.3 Component Ablation

Section 4.2 described the *classes of optimization* responsible for DAGSmith's gap above the baselines. This subsection asks how much each of DAGSmith's own architectural components contributes and how they interact. lin: Reference the subsection in Section 3 for each of the ablation. We ablate along two axes: the three optimization dimensions of LLM refactoring (R), materialization tuning (M), and frequency optimization (F); and the dataflow analysis that targets the LLM stage. We ablate on Tuva alone, since it is the larger and more structurally varied project and exercises all three dimensions meaningfully, whereas Stripe's single-run signal sits close to the run-to-run noise band, making fine-grained comparisons unreliable. lin: what is "single-run signal"? What is "run-to-run noise band"? Either be expliclt, or just remove this half of the sentence.

4.3.1 Three Optimization Dimensions. Table 4 reports the improvement of each subset of $\{R, M, F\}$ over the original Tuva project.

The numerics admit four observations, each reflecting a property of how the dimensions are designed to interact.

Slot reductions track total CPU work, elapsed tracks the critical path. lin: The paragraph head should summarize what we have learned about your approach (or different components), instead of which metrics track which. Recall that slot measures the total CPU-time the project consumes; elapsed measures the wall-clock time from first model to last. For every variant that includes refactoring or materialization, the slot reduction exceeds the elapsed reduction, because DAGSmith removes work parallel to the critical path more aggressively than work on it: consolidating many sibling models into one shared intermediate saves slot proportional to the sibling count but saves elapsed only the slowest sibling's

Table 4: Tuva: dimension ablation (% reduction over the original project, averaged across the three Tuva profiles in Table 2). R = refactoring, M = materialization tuning, F = frequency optimization. Bold marks the best variant in each column.

Variant	Elapsed	Slot
original	–	–
F only	13.2	11.0
M only	34.7	37.0
R only	40.0	49.1
M + F	31.0	54.0
R + M	**43.1**	54.7
R + M + F (full)	42.6	**67.7**

share. The gap is largest for R-only (49.1% slot vs. 40.0% elapsed). Frequency optimization is the exception, cutting elapsed and slot by comparable amounts (13.2% vs. 11.0%) because it removes whole scheduled executions rather than parallel work. This is why the full pipeline holds the best slot number but not the best elapsed: on top of R+M, F adds heavily to slot (54.7% to 67.7%) by removing off-critical-path runs while leaving elapsed flat (43.1% to 42.6%), a half-point difference within the run-to-run noise band against a 13-point slot gain that is F's intended effect.

R-only is close to R+M because R is not pure refactoring. When DAGSmith's refactoring stage introduces a new model, it also assigns that model an optimal materialization as part of producing the candidate (§??). lin: broken reference The R-only variant therefore already tunes newly created models; what M adds is project-wide retuning of pre-existing models, a smaller marginal change, so the gap between R-only (49.1%) and R+M (54.7%) is modest. M-only (37.0%) trails R-only because it cannot create models and can only revise materialization on the original DAG. lin: If M only adds marginally on top of R, shouldn't we just remove M from the pipeline and use R+F?

Refactoring and materialization are coupled. R+M (54.7%) exceeds the larger singleton (R: 49.1%) by 5.6 points: the refactored DAG admits a different optimal materialization than the original, and the joint pass finds savings neither stage produces in isolation. This makes the Section 4.2 argument concrete: pattern 4 of Section 4.2 contributes to R+M but to neither R nor M alone. lin: Eh this paragraph and the paragraph above actually conflicts with each other... this paragraph says that both stages are important, but the previous paragraph says that what M adds over R is only marginal change. So which is our conclusion? We're referencing exactly the same number here.

Frequency's increment depends on what came before it. lin: it's unclear what does "what" means here? Models? Improvements from other optimizations? We should be explicit. F alone delivers 11.0%, adds 17.0 points on top of M (54.0 vs. 37.0) and 13.0 on top of R+M (67.7 vs. 54.7). F's reach is bounded by the over-scheduled work present in the project, and that pool depends on the prior dimensions: refactoring can move work onto the right cadence and remove it from F's surface (splitting a fast-cadence model into a slow rollup and a fast lookup), while materialization tuning leaves the over-schedule pool largely intact, so F has more to remove on top of M than on top of R+M. The increment never collapses, which confirms F is structurally orthogonal to R and M rather than a redundant pass, and its size confirms the three dimensions interact in non-trivial ways. lin: size of what? time improvements?

Table 5: Dataflow Analysis: targeting ablation on Tuva (% slot reduction over the original project) under a fixed LLM-call budget across all arms. Whole-DAG refactoring produces no runnable project, so it has no cost to report (see text).

Selection rule	Single-run slot	Freq-weighted slot
Whole-DAG	no valid DAG	
Random subgraphs	43.1	58.0
Dataflow-guided (DAGSmith)	**51.1**	**67.7**

4.3.2 Dataflow Analysis. This experiment isolates the contribution of the dataflow analysis that produces DAGSmith's candidate set (Section 3.1): does structural targeting direct the LLM stage to more productive regions of the DAG than alternatives that do not use it? We hold the LLM-call budget fixed across all arms lin: what does "arms" mean, so the comparison reflects how well a fixed budget is spent rather than how many invocations are made. In the standard pipeline, dataflow analysis selects at most 20 subgraphs on Tuva, each capped at 60 nodes with 3 diagnostic-and-correction attempts; we hold this budget fixed across two alternatives. Table 5 reports the result.

Whole-DAG refactoring. The first alternative hands the entire project to the LLM under the same 3-attempt budget, testing whether bounding its attention to a region is needed at all. On Tuva the whole DAG fits in context (134k input tokens) and analysis succeeds, surfacing most of the opportunities our pipeline finds, but implementation never produces a runnable project. The response must restate every rewritten model in full, so it grows long enough to truncate mid-output and leave dangling references; cross-model references grow inconsistent as a fix to one model breaks an assumption in another; and because every broad attempt breaks somewhere, the correction loop retreats in scope across the three attempts (51, then 35, then 8 models). A broad rewrite captures most of the gain but touches too many interdependent models to verify, while one narrow enough to verify captures only a fraction. Dataflow analysis resolves this by making each subgraph small enough to verify while still reaching the high-value regions one at a time.

Random subgraphs. The second alternative changes only the selection rule, holding the subgraph count, size cap, and refactoring procedure fixed: we draw a model uniformly at random, take its parents, children, and siblings as a subgraph, and repeat until the count matches the standard pipeline, discarding any neighborhood over the node cap. This isolates the contribution of *where* the pipeline chooses to look. As Table 5 shows, random selection reaches 43.1% single-run and 58.0% frequency-weighted slot reduction, against 51.1% and 67.7% for dataflow-guided selection. The gap is a structural mismatch: an edge-based subgraph groups a model with its dependency neighbors, but the models worth refactoring together are elsewhere in the project, computing the same pattern over different inputs without sharing a single edge, so no edge-based

Table 6: Equivalence-verification outcomes on Tuva's 12 refactoring subgraphs (three-attempt budget each).

Outcome	Subgraphs
Pass within 1 attempt	25% (3/12)
Pass within 2 attempts	100% (12/12)
Never pass (budget exhausted)	0%

neighborhood contains both and widening it only adds irrelevant dependency-adjacent models. Dataflow-guided selection groups on the structure of the computation rather than on dependency edges, placing the models that compute the same pattern in one subgraph regardless of where they sit in the DAG.

4.4 System Characterization

lin: I actually feel like we may also want a table about how many models are there in each project, how many we refactored/tuned materialization, and what percentages are they for each project. Seems to be a helpful metric to gauge the impact of our refactoring.

This subsection asks whether the machinery behind the gains is sound and affordable: the equivalence-verification loop that catches unsafe rewrites, the LLM cost of running DAGSmith end to end, and the accuracy of the learned cost model that drives materialization.

4.4.1 Equivalence-Verification Effectiveness. The question for the verification loop (§3.2) is whether it catches unsafe rewrites without discarding safe ones, under a budget of three correction attempts per subgraph. Of the 20 candidate subgraphs on Tuva, 12 fit within the size cap and reach verification (Table 6). Only a quarter pass on the first attempt, and the rewrites the diagnostics reject fall into two recurring patterns a per-query checker cannot detect. The dominant one is a grain collapse: the rewrite reads a value from a pre-deduplicated sibling that keeps one row per key, silently dropping the secondary rows the original aggregates over (Figure 6). The second is algorithm substitution: a cheaper algorithm whose output differs from the original, such as a window-based interval merge that yields different groupings than the pairwise merge it replaces. Yet none of the 12 exhausts the budget: every rejected subgraph passes within a second attempt once its violated invariants are returned to the rewriter, so the loop is a gate rather than a dead end. On Stripe most candidates pass on the first attempt, since its smaller DAG and simpler SQL leave the rewrites less error-prone.

4.4.2 LLM Cost. Running DAGSmith is a one-time, design-time cost, paid once per project and amortized over every scheduled run of the optimized pipeline. Optimizing Tuva's 375 models with GPT-5.4 cost $11.51 and 2.4M tokens, about $0.03 per model, dominated by the rewriting stage (Table 7). This is small beside the recurring warehouse cost the optimized pipeline removes on every run.

4.4.3 Cost Model Accuracy. The learned cost model only needs to rank refactoring and materialization options within a project, not predict absolute runtimes or generalize across projects. Fit per project on the 156 instrumented Tuva models, its slot-time predictor, the quantity the ILP minimizes, reaches R^2=0.74 from static SQL and DAG features alone. Cardinality is harder (R^2=0.28), since row counts depend on data content that static features cannot see, but directional estimates are enough for ranking.

Table 7: End-to-end LLM cost of optimizing Tuva with GPT-5.4, by pipeline stage.

	Proposal	Filtering	Rewriting	Verification	Total
Cost ($)	2.00	0.96	5.68	2.88	11.51
Share (%)	17	8	49	25	100

5 RELATED WORK

Positioning. DAGSmith is, to the best of our knowledge, the first system to treat the explicit pipeline DAG as the optimization object, restructuring node boundaries and deciding refactoring, materialization, and refresh frequency jointly. The closest lines of work each hold one of these dimensions fixed.

Single-query SQL rewriting. Rule-based rewriters [26, 28], program-synthesis rewriters [10], and LLM-based rewriters [14, 15, 22] transform a query into an equivalent form before the database optimizer plans it; two of our baselines, LearnedRewrite [28] and GenRewrite [15], are of this kind. All of them optimize one statement in isolation and leave the surrounding dependency graph unchanged, so rewrites whose benefit depends on neighboring transformations, such as reusing logic across models or removing work that no downstream output consumes, fall outside their scope.

Multi-query optimization and view selection. Multi-query optimization [19, 20] and common-subexpression sharing [21, 27] reuse intermediate results across statements, and materialized-view selection [1, 13] precomputes chosen results and rewrites consumers to read them; our cross-model CSE baseline follows this line [21]. These techniques match on syntactic structure and select persistence among candidates that already exist. They do not recognize the same logical computation written in dissimilar SQL, as in Example 1, and they keep every output intact, sharing how results are computed but never which are needed; DAGSmith reads downstream demand to trim redundant computation and simplify the dependency graph (Example 2).

View maintenance and refresh scheduling. Incremental view maintenance [4] and streaming materialized-view systems [17, 25] keep derived results fresh by processing only the rows that change, which lowers the cost of each refresh. DAGSmith addresses freshness from the opposite direction: rather than cheapening a refresh, it aligns how often each node runs with how often its inputs change and its outputs are consumed, and it splits a node along a frequency boundary when slow-changing work is trapped on a fast cadence (Example 3). The two approaches are complementary and could compose.

Workflow orchestration. dbt [12], SQLMesh [23], Airflow [2], Dagster [5], Prefect [18], Argo [3], Luigi [16], and Databricks Workflows [6] express recurring transformations and their dependencies as a DAG, the setting DAGSmith targets. Their schedulers optimize execution order, invalidation, retries, and resource allocation, but they leave the SQL semantics of each node untouched, which is where DAGSmith's reductions originate.

6 CONCLUSION

Explicit SQL pipeline DAGs create an optimization setting that is richer than traditional one-query-at-a-time rewriting. Once dependencies, node contracts, materialization choices, and refresh behavior are visible, the optimizer can restructure the pipeline itself rather than merely improve individual statements. DAGSmith is a step toward that goal: it treats dependency-aware pipeline optimization as a joint problem over graph structure, materialization, and frequency, and combines structural analysis with LLM-guided refactoring and verification to explore that space.

The core claim is that recurring SQL pipeline DAGs expose whole-pipeline optimization opportunities that are invisible to query-local rewriting, and exploiting them requires a system designed for dependency-aware reasoning.

REFERENCES

[1] Sanjay Agrawal, Surajit Chaudhuri, and Vivek R. Narasayya. 2000. Automated Selection of Materialized Views and Indexes in SQL Databases. In *Proceedings of the 26th International Conference on Very Large Data Bases*. 496–505.

[2] Apache Airflow. 2026. DAGs. https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html. Accessed 2026-05-26.

[3] Argo Workflows. 2026. DAG. https://argo-workflows.readthedocs.io/en/latest/walk-through/dag/. Accessed 2026-05-26.

[4] José A. Blakeley, Per-Åke Larson, and Frank Wm. Tompa. 1986. Efficiently Updating Materialized Views. In *Proceedings of the 1986 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 61–71. https://doi.org/10.1145/16894.16861

[5] Dagster Labs. 2026. Software-defined assets. https://docs.dagster.io/. Accessed 2026-05-26.

[6] Databricks. 2026. Configure task dependencies. https://docs.databricks.com/aws/en/jobs/run-if. Accessed 2026-05-26.

[7] Databricks. 2026. What is the medallion lakehouse architecture? https://learn.microsoft.com/en-us/azure/databricks/lakehouse/medallion. Accessed 2026-05-26.

[8] dbt Labs. 2024. The next big step forwards for analytics engineering. https://www.getdbt.com/blog/analytics-engineering-next-step-forwards. Accessed 2026-05-26.

[9] dbt Labs. 2025. 2025 State of Analytics Engineering Report. https://www.getdbt.com/resources/reports/state-of-analytics-engineering-2025. Accessed 2026-05-26.

[10] Rui Dong, Jie Liu, Yuxuan Zhu, Barzan Mozafari, Xinyu Wang, and Cong Yan. 2023. SlabCity: Whole-Query Optimization using Program Synthesis. https://par.nsf.gov/servlets/purl/10451715.

[11] Tristan Handy. 2026. What, exactly, is dbt? https://www.getdbt.com/blog/what-exactly-is-dbt. Accessed 2026-05-30.

[12] Tristan Handy. 2026. What, Exactly, Is dbt? https://www.getdbt.com/blog/what-exactly-is-dbt. Accessed: 2026-05-30.

[13] Venky Harinarayan, Anand Rajaraman, and Jeffrey D. Ullman. 1996. Implementing Data Cubes Efficiently. In *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 205–216. https://doi.org/10.1145/233269.233333

[14] Zhaodonghui Li, Haitao Yuan, Huiming Wang, Gao Cong, and Lidong Bing. 2024. LLM-R2: A Large Language Model Enhanced Rule-based Rewrite System for Boosting Query Efficiency. *Proc. VLDB Endow.* 18, 1 (2024), 53–65. https://doi.org/10.14778/3696435.3696440

[15] Jie Liu and Barzan Mozafari. 2026. GenRewrite: Query Rewriting via Large Language Models. *Proceedings of the ACM on Management of Data* 4, 1 (SIGMOD (2026), 1–26.

[16] Luigi Contributors. 2026. Building workflows. https://luigi.readthedocs.io/en/stable/workflows.html. Accessed 2026-05-26.

[17] Frank McSherry. 2022. Materialize: A Platform for Building Scalable Event Based Systems. In *Proceedings of the 16th ACM International Conference on Distributed and Event-Based Systems*. Association for Computing Machinery, New York, NY, USA, 3. https://doi.org/10.1145/3524860.3544408

[18] Prefect Technologies. 2026. Tasks. https://docs.prefect.io/v3/concepts/tasks. Accessed 2026-05-26.

[19] Prasan Roy, S. Seshadri, S. Sudarshan, and Siddhesh Bhobe. 2000. Efficient and Extensible Algorithms for Multi Query Optimization. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 249–260. https://doi.org/10.1145/342009.335419

[20] Timos K. Sellis. 1988. Multiple-Query Optimization. *ACM Transactions on Database Systems* 13, 1 (1988), 23–52.

[21] Yasin N. Silva, Per-Åke Larson, and Jingren Zhou. 2012. Exploiting Common Subexpressions for Cloud Query Processing. In *IEEE 28th International Conference on Data Engineering (ICDE 2012), Washington, DC, USA (Arlington, Virginia), 1-5 April, 2012*, Anastasios Kementsietsidis and Marcos Antonio Vaz Salles (Eds.). IEEE Computer Society, 1337–1348. https://doi.org/10.1109/ICDE.2012.106

[22] Zhaoyan Sun, Xuanhe Zhou, Guoliang Li, Xiang Yu, Jianhua Feng, and Yong Zhang. 2025. R-Bot: An LLM-based Query Rewrite System. *Proc. VLDB Endow.* 18, 12 (2025), 5031–5044. https://doi.org/10.14778/3750601.3750625

[23] Tobiko Data. 2026. SQLMesh Overview. https://sqlmesh.readthedocs.io/en/stable/concepts/overview/. Accessed 2026-05-26.

[24] Adrian Vogelsgesang, Michael Haubenschild, Jan Finis, Alfons Kemper, Viktor Leis, Tobias Muehlbauer, Thomas Neumann, and Manuel Then. 2018. Get Real: How Benchmarks Fail to Represent the Real World. In *Proceedings of the Workshop on Testing Database Systems (DBTest '18)*. Association for Computing Machinery, New York, NY, USA, Article 4, 6 pages. https://doi.org/10.1145/3209950.3209952

[25] Yanghao Wang and Zhi Liu. 2022. A Sneak Peek at RisingWave: A Cloud-Native Streaming Database. In *Proceedings of the 16th ACM International Conference on Distributed and Event-Based Systems*. Association for Computing Machinery, New York, NY, USA, 190–193. https://doi.org/10.1145/3524860.3543284

[26] Zhaoguo Wang, Zhou Zhou, Yicun Yang, Haoran Ding, Gansen Hu, Ding Ding, Chuzhe Tang, Haibo Chen, and Jinyang Li. 2022. WeTune: Automatic Discovery and Verification of Query Rewrite Rules. In *Proceedings of the 2022 International Conference on Management of Data*. 94–107. https://doi.org/10.1145/3514221.3526125

[27] Jingren Zhou, Per-Åke Larson, Johann Christoph Freytag, and Wolfgang Lehner. 2007. Efficient exploitation of similar subexpressions for query processing. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, Beijing, China, June 12-14, 2007*, Chee Yong Chan, Beng Chin Ooi, and Aoying Zhou (Eds.). ACM, 533–544. https://doi.org/10.1145/1247480.1247540

[28] Xuanhe Zhou, Guoliang Li, Chengliang Chai, and Jianhua Feng. 2021. A Learned Query Rewrite System using Monte Carlo Tree Search. *Proc. VLDB Endow.* 15, 1 (2021), 46–58. https://doi.org/10.14778/3485450.3485456